Tutorial #1 - PONG

Introduction

Welcome to the first of our AppGameKit Studio tutorials, aimed at those of you who are just starting to learn about the programming of AppGameKit Studio Script (tier 1 BASIC). During these tutorials we'll be looking at a number of different game genres, learning about various common coding techniques that are often used in game making, and bringing them all together to create a number of simple game templates.

Each tutorial will introduce different elements of game coding, all of which are reusable in future projects, and you will find yourself including various elements many times in your own future projects. During the progress, you'll be guided through each of the steps clearly, and be encouraged to change various values and lines of code so that you can learn first-hand how simple changes can affect your code and dynamically alter your games.

The entry level for these tutorials expects some initial understanding of the students in the use of AppGameKit Studio. For more beginner level tutorials they can read the AppGameKit Studio Online Guides and watch the YouTube Videos.

Students are also encouraged to join the Official AppGameKit Studio forums. There's a newbie section where new users will be helped by many experienced and friendly developers who use AppGameKit Studio all the time.

Objectives

These tutorials aim to teach coding and game development using a combination of example led and action-based teaching. Students will learn from studying the code snippets, reading and understanding the comments and gain an additional understanding of each section through a number of simple exercises aimed at reinforcing each lesson and will help build student confidence by demonstrating the ease with which the code can be altered.

This first tutorial aims to introduce the concepts of basic coding and is designed to lay the groundwork of future developments. During this tutorial we will:

- Introduce **variables** and learn how to use them effectively
- Learn the power of **functions**
- Gain a basic understanding of **sprites** and **image** manipulation
- Learn how to read external **input** from the keyboard
- Teach the user about **text** objects, and learn how to use them

At any point, examples of almost all coding commands can be accessed from the IDE (integrated development environment) simple by either clicking directly to the left of the command and pressing F1 on the keyboard. The student should be encouraged to learn this method from the beginning as it will prove to be an invaluable reference guide and is a resource often used even by programmers who are very familiar with AppGameKit Studio.

Students can also access a full command reference from the IDE. By clicking Help on the top tool bar, students can gain access to a number of reference sources, including video tutorials, a list of useful keyboard shortcuts as well as the command reference.

Each section of code is available as a separate project which can be loaded into the IDE. You will see a file icon and a file name like this;

🗁**`Code example pong05.agc`**

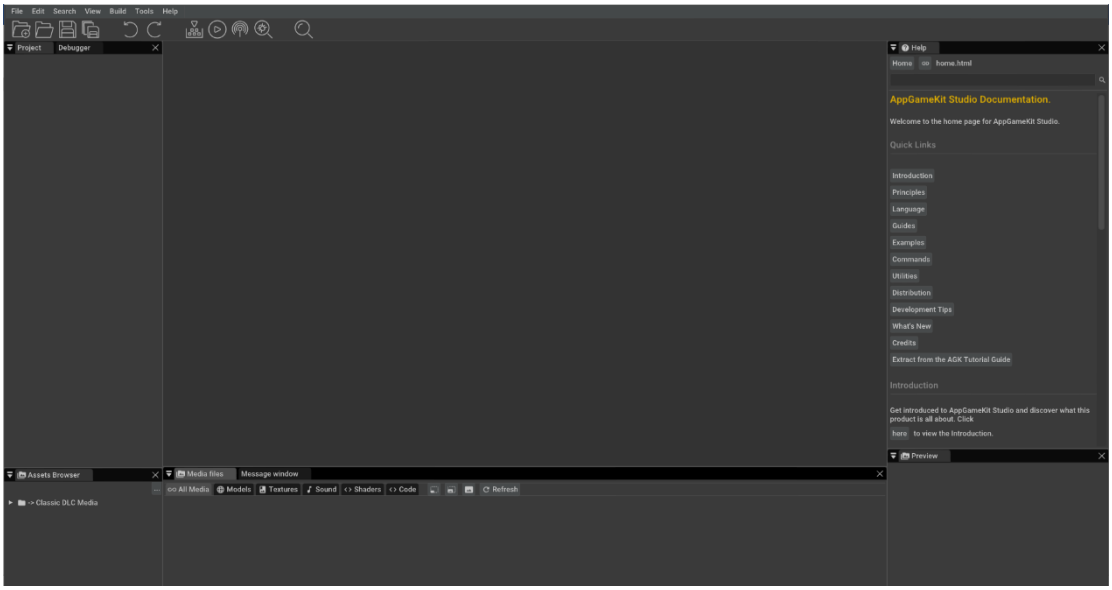
So if the code entered is not working due to typing errors the student can load in the project at this point in its development and carry on.

It's recommended that students are encouraged to learn to type the code into the IDE, as this will help to develop a familiarity with AppGameKit Studio overall and will promote confidence in using the functions of the IDE as well as reinforcing the learning process.

1. Getting Started with a new project

Our first tutorial will look at one of the very first games ever, the game of Pong. For those of you who don't know Pong was an arcade game based on table tennis and was originally released in 1972. It went on to become one of the first games to be developed for the very first dedicated gaming consoles. So, as you can see, we really are starting at the dawn of game making.

As these guides are aimed at the novice coder, let's start by learning how to set up a basic new project in AppGameKit Studio. Open AppGameKit Studio as normal (either via Steam or by clicking on the app in your chosen folder.). This will open the IDE:



The Main AppGameKit Studio IDE

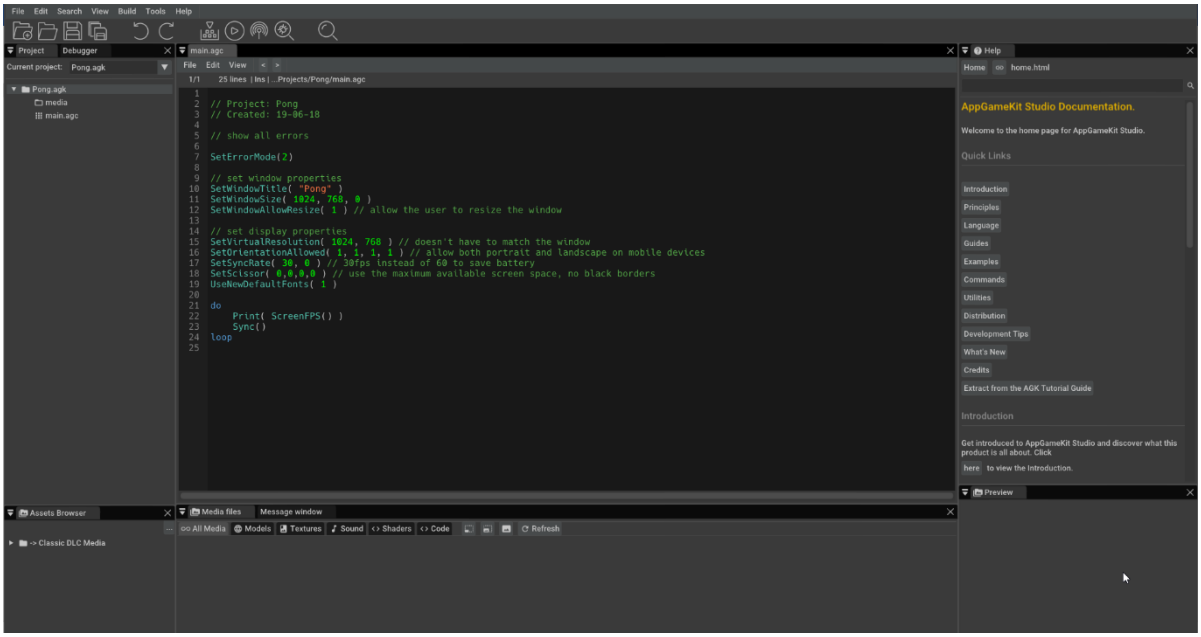
There's a lot you can do here, but we'll take you through it step by step. For now, we'll just look at creating a new project for our Pong Comment.

This is straight forward, and like many things in AppGameKit Studio, semi-automated to make your life easier. Move the mouse pointer to the File menu, select New Project and you will open a window in the middle of your screen.

New Project Dialogue Box

This box is where you create your project. In the box called Name: - this is where you type the name of your project. Below this is a field where you choose where you want the game code to be stored. Use the … button to choose a folder where you want the project files to be saved to. Make a note of this path, and this will also be where you will need to store any media for your project.

For now though, let's create our project by entering the name Pong into the name box, and clicking Create.



AppGameKit Studio will create some initial code to get you started

Some very basic code will be created automatically in the main IDE window. This is your coding window and is where you'll edit your code. The left hand window is your project information window and can be used to track your code and quickly access various elements of it, but for now, all we'll focus on is the centre window.

The code can be loaded from the file pong01.png.

For ease of understanding, we've replicated the code here:

```
// Project: pong
// Created: 19-06-18

// Show all errors
SetErrorMode(2)

// Set window properties
SetWindowTitle( "Pong" )
SetWindowSize( 1024, 768, 0 )
SetWindowAllowResize( 1 ) // Allow the user to resize the window

// set display properties
SetVirtualResolution( 1024, 768 ) // Doesn't have to match the window
SetOrientationAllowed( 1, 1, 1, 1 ) // Allow both portrait and landscape on mobile devices
SetSyncRate( 30, 0 ) // 30fps instead of 60 to save battery
SetScissor( 0,0,0,0 ) // Use the maximum available screen space, no black borders
UseNewDefaultFonts( 1 ) // Since version 2.0.22 we can use nicer default fonts

do
    Print( ScreenFPS() )
    Sync()
loop
```

This code is automatically saved by the IDE. You can also load this code from the tutorial folder under the filename: pong01.agc

There are already a number of coding tricks here that you'll use in your own projects, so let's take a look at the generated code.

One of the most useful things that has happened here, is the use of comments. A Comment is a note in your code that isn't part of the code, but a reminder of what is happening. These are amazingly useful things, and we really recommend adding comments wherever possible, so you or others can quickly understand your code.

There are two ways of adding a Comment. In the code above the use of // before any text will denote the start of a Comment. You may also use the word Rem. (Which is a shortening of the word Remark). Take a look at the code above and you'll see that several lines use comments to explain further what the code does.

As an example, we'll add our own Comment to the automated code. Let's add a comment below `//Project: pong` stating that this is a tutorial. Click on the end of the word pong and press Enter/Return to move down a line.

For our tutorial we'll add the comment `// An AppGameKit Studio tutorial`

Type or copy and paste the above into your code. The start of our code should now look like this.

```
// Project: pong
// An AppGameKit Studio tutorial
// Created: 2017-04-04
```

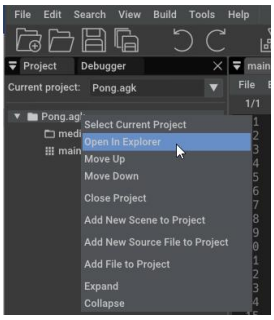
Simple steps, but well worth remembering to do. Well commented code (often referred to as Documented code) will save many hours of coding time as your projects grow.

Take a look at the code in your IDE and read the comments. These should help you understand what each command is doing. For additional help with commands including syntax, you can click on the beginning of any command and press F1. Do this now for `SetWindowSize( 1024, 768, 0 ).` This will bring up a separate help window with information about the command. Almost all commands have a similar help window and even experienced programmers find this invaluable. Try clicking on a few more to learn about them.

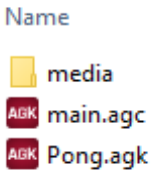
Once you've finished trying the help functions, now would be a good time to save your code. You can save your work from the File menu, click File and then click Save. You can also do this from the IDE by clicking the save icon on the top tool bar or by pressing Ctrl + S. All will have the same result.

This is very important and it's a really good habit to get into. Most coders save every few minutes and some between each build. Remembering to save your work frequently can avoid endless hours of frustration and work loss.

Together we've created a standard project and learnt a little about the IDE, let's take a quick look at the folder the code is stored in. If you remember you were asked to make a note of the folders location, hopefully you've done this and know where it is on your hard drive. If you cannot remember where it's located you can right click on the project name and a small menu will appear from which you can open the project folder's location in the file explorer:



Once you've found the folder and opened it, you should see something very much like this:



If the media folder is missing, you'll need to create one in the same folder as the Pong.agk and main.agc files.

This media folder is the most important part of your project, it is where you will store all of your images, 3D objects, sounds, music, videos and other files you'll want to use in your app.

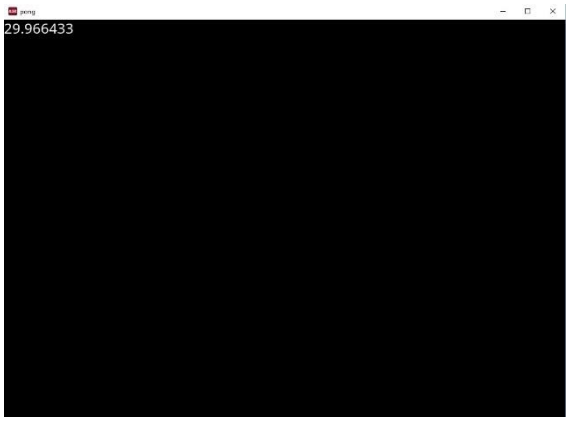
We'll need some media for our project. To help you along we've prepared all the media you'll need for this project for you. You can download a zipped version HERE. All you need to do is unzip the folder and copy the contents into the media folder for your Pong project. Remember you only want the contents, so either replace the media folder with this one, or copy the images and sound files into your own.

If everything has gone well, the contents of your media folder should look like this:

Media for your Pong game

Finally, let's just look at what this project looks like when it's compiled and run. You can do this very quickly by clicking the run icon.

The compile process should be very quick, but on first running can take a couple of seconds. If all has gone well, and we've not reason to think it hasn't you should see a window similar to this:



The number you see at the top right, is the frames per second (FPS). If you look at the code, we've restricted the FPS to 30 to save battery life on mobile devices. This is done using the command `SetSyncRate( 30, 0 )` .Try changing the value 30 to another numbers to see the affect it has – for example try 60fps. Eventually you'll find the maximum FPS capped by your own system, some top end machines will get 1000's frames per second, while lower end will reach the cap fairly quickly.

The value changes as the computers performance changes. This value is repeated because it is set in a `Do - Loop`. When the program encounters a `Loop` command it will revert back to a `Do` command, thus repeating everything in that loop. This is a great way to keep running your game loop. If you're wondering about the `Sync()` command, this instructs your program to redraw everything to the screen. This is best only called once per loop otherwise you will freeze the frame rate more times than necessary.

You can close the active window either by clicking the X or clicking the Run button a second time.

Once you've finished playing around with the number, change the FPS back to 30 and we'll move on.

2. A little about Variables

🗁Example pong02.agc

Let's look now at variables. A variable is a word that contains information that we will be able to change that will affect what happens in our game. That information can be varied, and can contain, Strings (text), Integers (whole numbers), Floats (decimal numbers) and quite a few other things as well, but for now, we'll only worry about those three. We'll also take a look the difference between `Local` and `Global` variables during this tutorial.

Let's make a small change to our code to see the effect of each of these.

Add this code just before the `Do` command

```
//Define 3 variables
// Global variables can be used by any part of your code.
// Local (or variables that aren't defined as Global) are treated as one off and aren't
shared by your
//Game code. For now we'll only be looking at Globals.
Global MyString as String
//Once defined, let's give each one a value
MyString="Hello" // String variables contain text. To define something as text enclose it
between ""
Global MyInteger as Integer
MyInteger=12345 // Numeric values don't need "" and are entered normally
Global MyFloat as Float
MyFloat=1.2345 // Floats accept floating point numbers.
```

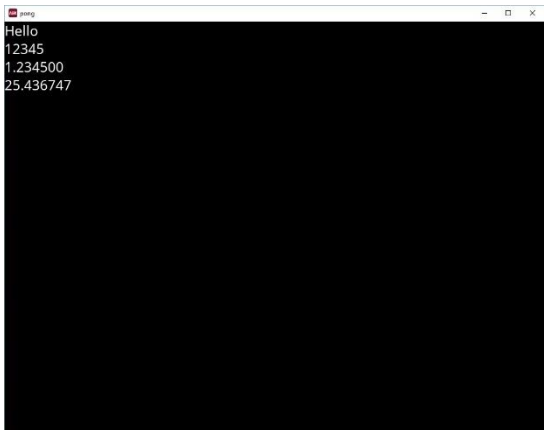
String variables can only contain text, while all numeric vairables can contain any number. However assigning a floating point number to an integer will round the number (remove the decimals places) and assigning an integer to a float will add a decimal point.

Now we've defined some variables, let's print them to the screen. Add the highlighted code to your project so it looks like this;

```
do

        Print(MyString)
        Print(MyInteger)
        Print(MyFloat)
        Print( ScreenFPS() )
        Sync()
loop
```

Ok, let's run this as we did before and look at the difference.



As you can see, our three variables are now displayed on screen. Have a go at changing these values to see the difference.

A variable doesn't just have to have a number, or a string attached. It can also be assigned a value from another variable.

Add this to your code just before the Do command

```
//New integer
Global MyInteger2 as Integer
MyInteger2=888
//Assign the value of MyInteger2 to Myinteger
MyInteger=MyInteger2
```

Run your code again, you'll notice that whatever value you assign to MyInteger at the start it will always equal the value you give to MyInteger2. This is because you are overriding the value of MyInterger using this line:

```
MyInteger=MyInteger2
```

This code can be reloaded from project pong03.agc.

EXERCISE*: If you think you've got the hang of that, why not try creating your own String variable and try assigning it to MyString?*

3. Custom Variables

🗁 **Example code pong04.agc**

Custom variables are possibly one of the most powerful and useful features of AppGameKit Studio coding. These are defined using the command Type. These allow a great deal of control over the content and make program flow easier to follow. This project can be loaded from pong04.agc.

Let's remove all of those variables we added as they aren't needed for our game and start some actual coding towards our end project. Once you've removed those additions your code should look like this.

```
// Project: pong
// An AppGameKit tutorial
// Created: 2017-04-04

// Show all errors
SetErrorMode(2)

// set window properties
SetWindowTitle( "pong" )
SetWindowSize( 1024, 768, 0 )
SetWindowAllowResize( 1 ) // Allow the user to resize the window

// Set display properties
SetVirtualResolution( 1024, 768 ) // Doesn't have to match the window
SetOrientationAllowed( 1, 1, 1, 1 ) // Allow both portrait and landscape on mobile devices
SetSyncRate( 30, 0 ) // 30fps instead of 60 to save battery
SetScissor( 0,0,0,0 ) // Use the maximum available screen space, no black borders
UseNewDefaultFonts( 1 ) // Since version 2.0.22 we can use nicer default fonts

do

    Print( ScreenFPS() )
    Sync()
loop
```

Let's add some custom variables, place this code before the Do command:

```
//Well set up our variables here
Type playerinfo
       //This will store the x&y position of the bat on screen, we'll use floats for more
       //Accurate positioning
       x as float
       y as float
       //Store the number of sprite will we be using
       sprite as integer
       //We'll also be allowing the speed to be changed.
       speed as float
       //and for coding reasons we'll need to know the direction the bat last moved in
       lastmove as integer
       //We're going to need to know how long it was since the last move
       lastmovetime as integer
       //As we may not have anyone to play with let's allow for a player to be the computer
       computer as integer
       //finallly, we'll need a score
       score as integer
EndType
```

As you can see, we've defined a custom Type variable called *playerinfo* which contains a number of what can be described as sub variables. There are several ways of using custom variables, but for now we'll be creating an array (a special variable with more than one element, you can think of it as a list or card index where each element can be referenced independently if you like, which is more or less what it is.). For those wondering why Dim is used rather than Array, the word Dim stands for Dimension.

```
//This is the last of the variables we'll be looking at today, the array, a very powerful way
//of referencing multiple values.
//In this case 2, one of each of the players.

Dim player[2] as playerinfo
```

You can learn more about Dim by clicking in front of the word Dim in the IDE and pressing F1

Now, if we run this it will be pretty dull again, as everything is happening behind the code with nothing to show on screen. So, let's add some a little eye candy by creating some sprites, and while we're at it, let's add some values to our array.

Add this to our code after the Dim statement

```
Global playerimage=1
// we're defining a global variable which we'll use to reference our bat image,
//you don't need to do this, but most coders find words easier to remember than numbers.
//load the image from our media folder, assigning it to the image number defined by
playerimage
//(in this case 1)
LoadImage(playerimage,"bat.png")
```

Now let's add some code to create some actual sprites and a little more information about the player. Add all of this after load image.

```
//Now to give some values to our array, let's start with player one, this will store their
//information in slot 1 of the array.
//these x and y positions will place player one 64 pixel in from the left hand side and in
// the middle of the screen vertically.
player[1].x=64
player[1].y=320
//we will be using sprite 10 for the players bat
player[1].sprite=10
//let's set a speed of 8 (this will move the bat 8 pixels at a time)
player[1].speed=8
//last move is already 0, as all new variables are set to a value of 0. We're simply doing it
// again to demonstrate adding values
player[1].lastmove=0
//is the player run by the computer? In this case a value of 0 says it is controlled by the
player.
player[1].computer=0
//let's also set the score to 0
player[1].score=0
//lastly let's actually make a sprite for the player bat and position it at position
player[1].x,player[1].y
CreateSprite(player[1].sprite,playerimage)
//Let's place it.
SetSpritePosition(player[1].sprite,player[1].x,player[1].y)
//and now let's do the same for player 2
player[2].x=916 // 64 pixels + the width of the image, which we know to be 44 pixels, in from
the right of the screen.
player[2].y=320
//we will be using sprite 11 for the players bat
player[2].sprite=11
//let's set a speed of 8 (this will move the bat 8 pixels at a time)
player[2].speed=8
//last move is already 0, as all new variables are set to a value of 0. We're simply doing it
again to
// demonstrate adding values
player[2].lastmove=0
//is the player run by the computer? In this case a value of 1 says it is controlled by the
```

```
computer.
player[2].computer=1
//let's also set the score to 0
player[2].score=0
//lastly let's actually make a sprite for the player bat and position it at position
player[2].x,player[2].y
CreateSprite(player[2].sprite,playerimage)
//now player 2 needs to have the bat facing the other way, and we've got a great command for
this.
SetSpriteFlip(player[2].sprite,1,0)
//Let's place it.
SetSpritePosition(player[2].sprite,player[2].x,player[2].y)
```

Lastly, let's make the bats a little easier to see by changing the background. As pong is a little like tennis, let's add a green background. There are many ways to do this, but for now, we'll load in a background image and create a sprite. We're sure that you can see how this is working as it is very similar to our previous sprites. You might notice that this one doesn't have any sprite positions. This is because we want it places at 0,0, which is the default position for newly created sprites.

```
//Let's make a background.
global background=3
Loadimage(background,"background.png")
Createsprite(background,background)
//This is another new command. Sprites can have layer, the higher the depth, the
//further back a sprite will be drawn. 100 will draw the sprite in behind other sprites.
SetSpriteDepth(background,100)
```

Now, save your code, click run and you should actually see something that looks a little more like a game.

Read through the code so far and try changing the positions of the bats by altering the x and y values stored in the player array.

E.G.

```
Player[1].x=500
Player[1].y=320
```

This will place the first player bat near the middle of the screen. Learn more about each command we've used by reading the comments and clicking in front of them and pressing F1.

Once you've confident that you understand what we've done so far, return any values you've changed back so that it matches the code above and we'll move on.

We're now going to look at another very important command. The Function.

A Function is a special bit of named code that is called from your main game loop. We use Functions if we want to do the same thing many times, or want to add something that AppGameKit Studio doesn't have a command for. In many ways a Function can be considered to be a sub program or a way of adding extra commands to AppGameKit Studio.

We're going to add our Functions at the end of our code. Let's add one that will control the player movement.

4. Adding movement

🗁Code example pong05.agc

```
//We pass the number of the bat we want to control to this function
//Which is an example of a local variable, and only applies to this Function, if you tried to
find out it's // value outside of this Function
//You'll receive an error.

Function controlbats(which)

  // Let's make sure that we only use bats that exist
  // We will be using a series of If - EndIF loops here
  // If is a conditional statement, and is either True or False.
  // When the True/False test is passed any code between the If and the EndIf will be run

  // Is the value of which (our local variable) equal 1 or 2 (our only possible bats)

  If which=1 or which=2
        // Is the bat controlled by the player. Remember we set player[?].computer to 0 if
        // it is. If it is, then run this If-Endif loop
        if player[which].computer=0
                // Let's use the arrow keys on the keyboard to move the player up and down
                // We can read the keyboard using a special command for devices with a
                // keyboard called GetRawKeyState(?) - We replace the ? with the scancode
                // of the key we want to read.
                // If the up arrow (scancode 38) is pressed
                If GetRawKeyState(38)=1
                        // Decrease the Y (vertical) value of the bat, that is up the screen
                        dec player[which].y,player[which].speed
                        // Mark the player as last having move negatively (up the screen)
                        player[which].lastmove=-2
                        // We'll record the time it happened here
                        player[which].lastmovetime=GetMilliSeconds()
                Endif

                // If the down arrow (scancode 40) is pressed
                If GetRawKeyState(40)=1
                        // Increase the Y (vertical) value of the bat, that is down the screen
                        inc player[which].y,player[which].speed
                        // Mark the player as last having move possitively (down the screen)
                        player[which].lastmove=2
                        // We'll record the time it happened here
                        player[which].lastmovetime=GetMilliSeconds()
                Endif

        endif

        // Is the bat controlled by the computer? Remember we set player[?].computer to 1
        // If it is then run this If-Endif loop
        if player[which].computer=1
        endif

        // Now lets' make sure that the bat can't leave the screen

        // If player[?].y is less than 8,
        If player[which].y<8
                // Set it to 8
                player[which].y=8
        endif

        // If player[?].y is greater than 632
        If player[which].y>632
                // Set it to 632
                player[which].y=632
        endif

        // Now let's update the actual sprite postion with it's new cordinated
        SetSpritePosition(player[which].sprite,player[which].x,player[which].y)

        // So we can calculate some bounce from the bat a little later,
        // We'll check how long ago it was since the bat moved
        // If it's more than half a second ago, let's forget the bat moved.
        if GetMilliseconds()-player[which].lastmovetime>500
                player[which].lastmove=0
        endif
```

```
    Endif

EndFunction
```

NOTE: As an aside you will find this link very useful : https://www.appgamekit.com/documentation/guides/scancodes.htm this will list all of the available scancodes and is an excellent reference.

Now, this all this code all very well, but if you run this, nothing much will happen as the code doesn't ever call that Function. So, to make sure that happens add this before the Sync() command.

```
    // Now let's look at another kind of loop
    // The For - Next loop isn't conditional
    // This will run through the numbers specified, in this case 1 to 2
    For checkbothbats=1 to 2
        // Pass the value that the loop is currently at to controlbats()
        controlbats(checkbothbats)
        // The next commands, updates the loop, and if it hasn't reached its target it will
run
        // All the commands between the For and related Next
    Next checkbothbats
```

SAVE NOW!

Ok, now all that code is in place, run it and press the up and down arrows. The bat to the left will move. You can play around with a lot of values now, such as the bat speed, and which bat it controls.

EXERCISE: *Try altering player[1].speed to another value, or setting player[1].computer to 1 and setting player[2].computer=0 and see what happens.*

5. Pong isn't much use without a ball!

🗁**Code example pong06.agc.**

So, we've got a bat that moves up and down.. Fun isn't it.. Well, no, not much, let's be honest. So it's time to add a ball. First let's set up a new Type.

We'll add this after the playerinfo Type after the command EndType

```
Type ballinfo
        x as float
        y as float
        sprite as float
        speed as float
        //these two are slightly different as they will control which direction the ball is
moving in
        movex as float
        movey as float
EndType

// Notice how we can create custom variables for normal variables as well as arrays?
Global ball as ballinfo
// We'll use sprite and image 2 for the ball. Sprites and Images can share the same value.
ball.sprite=2
LoadImage(ball.sprite,"ball.png")
CreateSprite(ball.sprite,ball.sprite)
// Set it's position
ball.x=512
ball.y=340
// Set the ball sprite at x and y
SetSpritePosition(ball.sprite,ball.x,ball.y)
// Set it's movement direction
// No up or down movement for now
ball.movey=0
// We introduce another useful command here, RandomSign has a 50/50 chance of creating a //
//Negative version of the number
// In the brackets, in this case it will either produce -1 or 1.
// This will create a 50/50 chance of the ball being moved left or right at first
ball.movex=RandomSign(1)
// Lastly let's have some speedy movement
ball.speed=4
```

Now, if you run this code, you'll see the ball in the middle of the screen, but not much else, so now let's create a new Function that will move the ball around. Add this at the end of the code. We aren't passing any values to this function, so () is empty.

```
Function moveball()

        // Add the value of ball.movex and ball.movey to the balls position
        // We multiply it by the balls speed so it moves quickly
        // Remember we can change the speed of the ball.
        ball.x=ball.x+ball.movex*ball.speed
        ball.y=ball.y+ball.movey*ball.speed
        // Now let's update the position of the ball sprite.
        SetSpritePosition(ball.sprite,ball.x,ball.y)

EndFunction
```

And of course, we'll need to call this Function won't we? So add a call before the Sync() command in the main Do-Loop.

```
// Let's more the ball
moveball()
```

EXERCISE: *Try running this code a few times and note how the ball changes direction randomly during each run. Remember why this is? Try changing some values to only make the ball move left, then only right and then back to random movement.*

6. Making a game

🗁Code example pong07.agc

So, we've got a lot going on, but we're still a little way away from a game but we are getting closer. Let's see about bouncing the ball off of the bats, that will make it more playable. For this we will need to use a combination of sprite collision technique and a tiny bit of maths.

We'll be changing two Functions for this routine:

Controlbats()

and

Moveball()

First let's check to see if the ball has hit a bat. We'll do this by adding this code to the Controlbats() function just before the last EndIf

```
// Time to look at some collisions.
// We'll compare each bat to see if it's hit the ball
// GetSpriteCollision will return 0 if the two sprites have not collided
// and 1 if they have.
If GetSpriteCollision(player[which].sprite,ball.sprite)=1
        // By seeying ball.movex to a negative of it's own value, we effectively
        // reverse it's direction
        ball.movex=-ball.movex
        // Now let's see if the bat was moving, we're simulate a little physics here
        If player[which].lastmove<>0
                ball.movey=player[which].lastmove
        Endif
Endif
```

Now, we'll back a couple of changes to the moveball() Function, this time to see if we've bounced off of the side of the screen. You'll notice here the use of 2 checks in the same line. You can use a number of boolen maths commands. At the moment, we're only interested in OR with means, both being correct is valid. Add this code before the EndFunction.

```
// We also need to see if the ball has bounced of the side of the screen by checking the
// ball's Y position. We can do this easily by checking if it's near the edge
// and we only need to reverse it's Y move just as we did with
// the X move just know

        if ball.y<8 or ball.y>740
                ball.movey=-ball.movey
        endif
```

7. Let's look at some basic AI (Artificial Intelligence)

🗁Code example pong08.agc

We do mean basic here, for this kind of game, all the computer needs to do is track the movement of the ball and move up or down so that it is on a collision course. We'll make a new function for this, which we'll be calling from our Controlbats() Function. Our new Function is called Computermove()

```
Function computermove(which)

        // All we're doing here is tracking the ball
        // We'll do this by comparing the bat position Y (vertical)
        // with the ball Y (Vertical). If the player bat below the ball (greater than)
        If player[which].y>ball.y
                // Decrease the Y (vertical) value of the bat, that is up the screen
                Dec player[which].y,player[which].speed
                // Mark the player as last having move negatively (up the screen)
                player[which].lastmove=-2
                // We'll record the time it happened here
                player[which].lastmovetime=GetMilliSeconds()
        Endif

        // If the player bat above the ball (less than)
        If player[which].y<ball.y
                // Increase the Y (vertical) value of the bat, that is down the screen
                Inc player[which].y,player[which].speed
                // Mark the player as last having move possitively (down the screen)
                player[which].lastmove=2
                // We'll record the time it happened here
                player[which].lastmovetime=GetMilliSeconds()
        Endif

Endfunction
```

If we want to make sure that the computer tries to bat back the ball we need to make another addition to the code. We obviously need to call the computermove() Function. We don't think we'll be surprising anyone if we say the correct place to call the Function is from the controlbats() Function, to do this, we need to place this code:

```
// If its the computer, call the computermove(?) Function.
// The code will return here afterwards.
computermove(which)
```

after the line that reads:

```
if player[which].computer=1
```

EXERCISE: *Play around with the settings to see if you can make the computer play itself. You'll actually only need to change one number to do this. See if you can work out where.*

8. It's all about winning

🗁 **Code example pong08.agc**

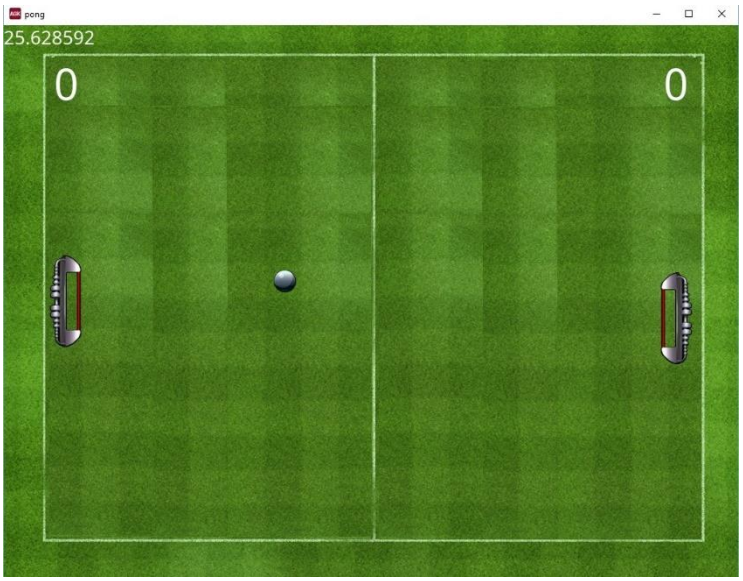
For our final instalment of this tutorial, we'll need to find a winner. One player will win if their opponent misses the ball 10 times. So, we'll need to track this and display the results. For this we're going to need a new type of object, called a text object. Like sprites and images, a text object must have a unique ID, but this ID needs only to be unique to text objects, so you can have a sprite, a text object and an image all referenced by the number 1 or a variable with the value of 1.

We need to set up the text, so we'll do that at the start, just before our main Do – Loop.

```
// Let's create a text object that will display the score.
CreateText(1,"0")
// Just like sprites, we can assign it a screen position
SetTextPosition(1,50,20)
// We want this text to be quite large, so we'll give it a size of 80
SetTextSize(1,80)
// Let's do it all again for a second object.
CreateText(2,"0")
SetTextPosition(2,920,20)
SetTextSize(2,80)
```

Don't forget, if you want to know more about these commands, you can select any of them and press F1

SAVE your code and click Run again, and you should see a screen very similar to this.



9. Finishing off

🗁 **Code example pong09.agc**

You'd almost think we had a game here. We've just a couple of minor things left to complete this project. First, let's set it up so that if the ball leaves the screen, someone scores a point and the ball is reset.

The best place to add this code is moveball(), so let's do that. Add this to the end of the moveball() function code, before the EndFunction.

```
// Let's see if someone has won a point.
// We've check the balls position to see if it's offscreen.
// We'll also make a local variable to see if we need to reset the ball.
// We're also keeping the comments limited here, see if you can work it out.

Local hasballleft=0

If ball.x>1056
        hasballleft=1
        inc player[1].score,1
Endif

If ball.x<-32
        hasballleft=1
        inc player[2].score,1
Endif

// We need to know if a player has won
// This happens when one player reaches 10
// So, let's check for that.
If player[1].score=10 or player[2].score=10
        // Reset everthing, effectively reseting the game
        hasballleft=1
        player[1].score=0
        player[2].score=0
Endif


// We'll also need to reset the ball postion if it's left the screen.
if hasballleft=1
        //Set it's position
        ball.x=512
        ball.y=340
        //Set the ball sprite at x and y
        SetSpritePosition(ball.sprite,ball.x,ball.y)
        //Set it's movement direction
        ball.movex=RandomSign(1)
        // No up or downmovement for now
        ball.movey=0
endif
```

We've left most of the comments out here, but we think you'll be able to work out what's going on. Now, only one more thing remains to ensure that we've got a game, and that is to update the score and declare a winner. We're going to introduce two final new commands here, one is the STR command. You may recall that we said earlier that a STRING could only contain text and that a INTEGER or a FLOAT could only contain numbers. We'll that's true, but the STR command converts a numeric value into a STRING. Clever eh?

The second is SetTextString. This command lets us alter the text of a text object we've already created. This means we can update the text objects that contain our scores with the new scores.

The best place for this would be in a new Function. Let's call it scores() and let's add it at the end of the code.

```
Function score()

        //You might be wondering what kind of variable this is.
        //Well, it's a String variable. We can simply add the symbol $ to the end of a
        // variable to create a String.
        Local change$
        //Not let's assign the score to it, using Str
        // to convert the numeric score to a string.
        change$=str(player[1].score)
        //Finally let's alter the text to contain the new string.
        SetTextString(1,change$)
        //Let's do it again for the text object 2.
        change$=str(player[2].score)
        SetTextString(2,change$)

EndFunction
```

Now, as before, a function is useless if we don't call it, so let's do just that. The best place for this is in the main game Do-Loop, just before the Sync() command.

So, add this there.

```
// Update score
score()
```

Well, that's it so for this tutorial. There is certainly a lot more that we could have added, and we'll be looking at things like adding sound effects and a menu for later tutorials, but for now we really hope that you've learnt from this and that it's inspired you to make some changes yourself to learn more about coding with AppGameKit Studio.